
Calculus for Machine Learning

Refresher Module 3, Part 1 — Derivatives, Gradients & Optimisation

the derivative · the gradient & Jacobian · the total differential · Taylor · the losses you differentiate · the chain rule & backprop · gradient descent · autodiff

Computing, Robotics & Bionics

3rd-year Bioengineering — companion to A. Géron, *Hands-On Machine Learning with Scikit-Learn and PyTorch*

*This is the third of four refresher modules, split into two parts. **Part 1** (this document) develops just enough differential calculus to understand how machines learn: the derivative and the chain rule, the gradient and the Jacobian, the chain rule run through a path into a scalar field — the total differential of physics — Taylor expansions as the local models that optimisation actually minimises, the three losses you will actually differentiate and the one gradient shape they share, why depth makes gradients vanish or explode, gradient descent, the algorithm that trains almost every model, and automatic differentiation, the engine inside PyTorch. **Part 2**, taught later alongside the Control Theory unit, treats integration as accumulation and cashes it out in the integral term of a PID controller. Part 1 is **self-contained**: together with its companion Colab notebook — a bioengineering-adapted superset of Géron's `math_differential_calculus` and autodiff material — it is your complete reference for this half. The Jacobian here is the manipulator Jacobian of the Robotics unit; the gradient powers Géron Ch. 4 and Ch. 9–11. Each section ends with exercises.*

Prerequisites: Modules 1–2 (NumPy, linear algebra); high-school calculus is helpful but not assumed. Software: a web browser and a Google account (everything runs in Google Colab; nothing to install).

Contents

1	The Derivative	2
1.1	Slope, limit, rate	2
1.2	The rules you actually use	2
1.3	A worked derivative	2
1.4	The classification losses: log-loss and softmax	3
2	Numerical Derivatives	4
3	Functions of Many Variables	4
3.1	Partial derivatives and the gradient	4
3.2	Along a Path: the Chain Rule and the Total Differential	4
3.3	Why the Gradient Points Uphill — and the Tangent Plane	6
3.4	The Jacobian	9
3.5	The Hessian	9
4	Taylor Expansions: Local Polynomial Models	10
4.1	One variable	10
4.2	What it says about finite differences	10
4.3	Many variables	11
5	The Chain Rule, Computational Graphs and Backpropagation	12
5.1	A real layer: differentiating with respect to a matrix	12
5.2	Why depth is hard: vanishing and exploding gradients	13
5.3	Backprop is a product of Jacobians — and why it runs backwards	14
6	Optimisation: Gradient Descent	15
6.1	The learning-rate law	16
7	Automatic Differentiation and PyTorch	17
8	The Lab, and Where This Leads in Géron	18

1 The Derivative

Training a model means **minimising a loss**, and minimisation needs slopes. The derivative is the central tool of this module.

1.1 Slope, limit, rate

The derivative of f at x is the slope of its graph there — the limit of the secant slope as the two points merge:

$$f'(x) = \frac{df}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}. \quad (1.1)$$

Read it as a **rate of change**: how fast the output moves per unit of input. In a model, f is the loss and x a parameter, so $f'(x)$ says which way and how fast the loss changes if we nudge the parameter — exactly the information needed to improve it.

1.2 The rules you actually use

Rule	Statement	Note
Power	$(x^n)' = nx^{n-1}$	the workhorse
Sum	$(f+g)' = f' + g'$	differentiate term by term
Product	$(fg)' = f'g + fg'$	
Quotient	$(f/g)' = \frac{f'g - fg'}{g^2}$	
Chain	$f(g(x))' = f'(g(x)) g'(x)$	the key to backprop

A few derivatives recur throughout ML: $(e^x)' = e^x$, $(\ln x)' = 1/x$, $(\sin x)' = \cos x$, and for the logistic (sigmoid) $\sigma(x) = \frac{1}{1+e^{-x}}$ the tidy $\sigma'(x) = \sigma(x)(1 - \sigma(x))$.

Key idea

The chain rule is the most important rule in this module. A neural network is a deep composition $f_L(\cdots f_2(f_1(x)))$, and training it means differentiating that composition — which the chain rule does layer by layer. **Backpropagation** is the chain rule applied to a computational graph, nothing more.

1.3 A worked derivative

Derivative of a squared-error loss

For one sample, mean-squared error in a parameter w is $L(w) = \frac{1}{2}(wx - y)^2$. By the chain rule, with inner $u = wx - y$ ($du/dw = x$) and outer $\frac{1}{2}u^2$ ($d/du = u$),

$$\frac{dL}{dw} = (wx - y) \cdot x.$$

This single derivative is the gradient that linear regression and a neuron descend. Notice it is the error $(wx - y)$ times the input x — a pattern you will see again and again.

1.4 The classification losses: log-loss and softmax

Squared error is the loss for regression. For *classification* the model outputs a probability and is trained on a different loss — and the promise that the pattern repeats is worth cashing in immediately, because the algebra looks alarming and collapses to almost nothing.

A binary classifier computes a score $z = \mathbf{w}^T \mathbf{x} + b$ and squashes it with the logistic function of Section 1.2, $\hat{y} = \sigma(z)$, which is read as the probability of the positive class. The loss is the **log-loss** (or **binary cross-entropy**), which punishes confident mistakes:

$$L = -[y \ln \hat{y} + (1 - y) \ln(1 - \hat{y})], \quad \hat{y} = \sigma(z). \quad (1.2)$$

Derivative of the log-loss

Differentiate (1.2) through the sigmoid in two steps. The outer part gives $\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}$, and the inner part is the tidy derivative from Section 1.2, $\frac{d\hat{y}}{dz} = \sigma'(z) = \hat{y}(1-\hat{y})$. Multiplying them, the denominators cancel completely:

$$\frac{\partial L}{\partial z} = \left(-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}\right) \hat{y}(1-\hat{y}) = -y(1-\hat{y}) + (1-y)\hat{y} = \hat{y} - y,$$

and one more step of the chain rule ($\partial z / \partial \mathbf{w} = \mathbf{x}$) gives

$$\frac{\partial L}{\partial \mathbf{w}} = (\hat{y} - y) \mathbf{x}.$$

Error times input again — the same shape as the squared-error gradient above, with the error now measured as “predicted probability minus true label”. The sigmoid and the logarithm were built for each other: each one’s awkwardness cancels the other’s.

For more than two classes the same thing happens one dimension up. A score z_k per class is turned into probabilities by the **softmax**,

$$\hat{y}_k = \frac{e^{z_k}}{\sum_j e^{z_j}}, \quad (1.3)$$

and trained with the **cross-entropy** $L = -\sum_k y_k \ln \hat{y}_k$ against a one-hot label. Grinding through the quotient rule, all the cross terms cancel and what survives is, in vector form,

$$\frac{\partial L}{\partial \mathbf{z}} = \hat{\mathbf{y}} - \mathbf{y}. \quad (1.4)$$

Key idea

Three losses, one gradient shape. Squared error gives $(\hat{y} - y)x$, log-loss gives $(\hat{y} - y)\mathbf{x}$, softmax cross-entropy gives $(\hat{\mathbf{y}} - \mathbf{y})$ at the scores — always *how wrong you were*, times *what went in*. This is why frameworks fuse the softmax into the loss function (**CrossEntropyLoss** takes raw scores, not probabilities): the fused gradient is a subtraction, so computing it separately would be slower and numerically worse. Logistic and softmax regression are Géron’s Chapter 4, the pair is the default output layer and loss of every classifier in Chapters 9–11, and the softmax of (1.3) reappears as the core of attention in the transformer chapters.

2 Numerical Derivatives

Often f is a piece of code, not a formula. We can still estimate its slope by **finite differences**, evaluating (1.1) at a small but finite h :

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} \quad (\text{forward}), \quad f'(x) \approx \frac{f(x+h) - f(x-h)}{2h} \quad (\text{central}). \quad (2.1)$$

The central difference is far more accurate (its error shrinks like h^2 rather than h). Too large an h gives truncation error; too small an h gives floating-point round-off, so there is a sweet spot around $h \sim 10^{-5}$ for central differences in double precision. Section 4 derives both halves of that trade-off.

Key idea

Gradient checking. Before trusting an analytic gradient (or a custom layer), compare it against a central finite difference. If they disagree beyond a small tolerance, the analytic gradient has a bug. This is a standard, life-saving habit when implementing models, and you will do it in the lab.

3 Functions of Many Variables

A model has many parameters, so its loss is a function of a **vector**. We differentiate with respect to each one.

3.1 Partial derivatives and the gradient

A **partial derivative** $\partial f / \partial x_i$ is the ordinary derivative in one variable with the others held fixed. Collecting them gives the **gradient**, a vector:

$$\nabla f(\mathbf{x}) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)^\top. \quad (3.1)$$

Its geometry is the whole point: the gradient points in the direction of steepest increase of f , and its negative points downhill. Its length is the steepness. Level sets (contours) of f are everywhere perpendicular to the gradient. This is why optimisation walks in the direction $-\nabla f$. None of that is obvious from (3.1), which is just a list of partial derivatives; Section 3.3 proves it.

3.2 Along a Path: the Chain Rule and the Total Differential

The gradient earns its keep the moment something *moves*. Physics is full of the same three-layer pattern: a scalar parameter feeds a vector, and the vector feeds a scalar. A particle's position is a vector function of time,

$$\mathbf{r}(t) = (x(t), y(t), z(t))^\top, \quad (3.2)$$

and a **scalar field** — an electric or gravitational potential V , a temperature, a drug concentration — assigns a number to every point of space, $V(\mathbf{r}) = V(x, y, z)$. Composing the two gives the potential the particle *experiences*, $V(\mathbf{r}(t))$: a scalar function of the single parameter t . Figure 1 draws the chain and its geometry.

Differentiating that composition is the chain rule, applied once per coordinate and summed, because t reaches V through all three of them:

$$\frac{dV}{dt} = \frac{\partial V}{\partial x} \frac{dx}{dt} + \frac{\partial V}{\partial y} \frac{dy}{dt} + \frac{\partial V}{\partial z} \frac{dz}{dt} = \nabla V \cdot \frac{d\mathbf{r}}{dt}. \quad (3.3)$$

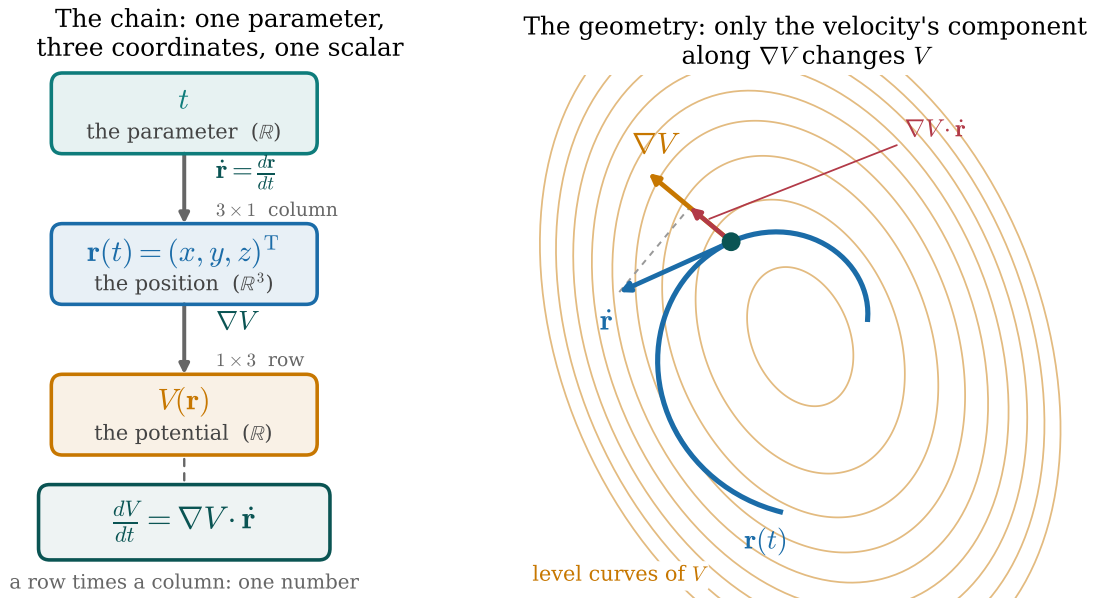


Figure 1: Left: the chain $t \mapsto \mathbf{r}(t) \mapsto V$. A scalar parameter feeds a vector of three coordinates, which feeds one scalar; the derivative of each link is written beside it, and the two multiply — a 1×3 row times a 3×1 column — to give the single number dV/dt . Right: the geometry in the plane. The particle rides the path $\mathbf{r}(t)$ with velocity $\dot{\mathbf{r}}$ tangent to it, while ∇V points across the level curves of V , uphill. Only the component of the velocity *along* the gradient changes V : sliding along a contour ($\dot{\mathbf{r}} \perp \nabla V$) leaves the potential untouched.

The compact form on the right is the same statement read as linear algebra: the gradient (3.1) is the derivative of the outer link, the velocity $\dot{\mathbf{r}} = d\mathbf{r}/dt$ is the derivative of the inner one, and the chain rule multiplies them. That is exactly the Jacobian product of Section 3.4, in its smallest interesting case: a 1×3 row times a 3×1 column giving a 1×1 result.

Now multiply (3.3) through by dt . The parameter disappears and what is left is the relation physicists write directly, the **total differential**:

$$dV = \frac{\partial V}{\partial x} dx + \frac{\partial V}{\partial y} dy + \frac{\partial V}{\partial z} dz = \nabla V \cdot d\mathbf{r}. \quad (3.4)$$

Read it as a first-order budget: each coordinate contributes its own partial derivative times its own displacement, and the contributions simply add. Crucially, no path and no parameter appear in (3.4) — t was scaffolding. The total differential relates an *arbitrary* infinitesimal displacement $d\mathbf{r}$ to the change it produces in V , whatever curve that displacement happens to belong to and however fast it is traversed. Choose a particular path and divide by dt and you are back at (3.3).

Three readings of (3.4) are worth having in mind:

- **Move along a contour and nothing happens.** If $d\mathbf{r}$ is perpendicular to ∇V then $dV = 0$: the displacement lies in a level surface (an equipotential), which is precisely the statement from Section 3.1 that level sets are perpendicular to the gradient. A satellite coasting along an equipotential neither gains nor loses potential energy.
- **Steepest ascent, quantified.** Write $d\mathbf{r} = \hat{\mathbf{u}} ds$ for a unit direction $\hat{\mathbf{u}}$ and a small distance ds . Then $dV/ds = \nabla V \cdot \hat{\mathbf{u}}$, the **directional derivative**: largest when $\hat{\mathbf{u}}$ points along ∇V , most negative when it points against. Gradient descent (Section 6) is nothing but this sentence, acted on.

- **Work and conservative forces.** For a force derived from a potential, $\mathbf{F} = -\nabla V$, the work done over $d\mathbf{r}$ is $dW = \mathbf{F} \cdot d\mathbf{r} = -dV$. The energy bookkeeping of mechanics is the total differential with a sign.

Key idea

The pattern *scalar* $\rightarrow$ *vector* $\rightarrow$ *scalar* is not a physics curiosity; it is the shape of every trained model. Replace t by a parameter w , $\mathbf{r}(t)$ by the vector of activations a layer produces, and V by the loss, and (3.3) becomes the backpropagation update of Section 5: sum over every route by which the parameter reaches the loss, each route contributing a product of local derivatives. The chain rule for a composition is always a product of Jacobians; physics runs it forwards along a path, and training runs it backwards from the loss.

3.3 Why the Gradient Points Uphill — and the Tangent Plane

Steepest ascent is a statement about a cosine

Take a unit direction $\hat{\mathbf{u}}$ in the domain and step a small distance ds along it. By the total differential (3.4) the change in f is $df = \nabla f \cdot (\hat{\mathbf{u}} ds)$, so the rate of climb in that direction — the **directional derivative** — is

$$\frac{df}{ds} = \nabla f \cdot \hat{\mathbf{u}} = \|\nabla f\| \cos \theta, \quad (3.5)$$

where θ is the angle between $\hat{\mathbf{u}}$ and ∇f , since $\|\hat{\mathbf{u}}\| = 1$. Every claim of Section 3.1 is now just a value of that cosine:

- $\theta = 0$ — step along ∇f and the rate is $\|\nabla f\|$, the largest value $\cos \theta$ allows. So the gradient's *direction* is steepest ascent and its *length* is that steepest rate. Both halves of the sentence come from the same formula.
- $\theta = 90^\circ$ — step perpendicular to ∇f and the rate is zero: f does not change, so you are moving along a contour. That is why level sets are perpendicular to the gradient.
- $\theta = 180^\circ$ — the rate is $-\|\nabla f\|$, the most negative available: steepest descent, along $-\nabla f$. This is the entire justification for the update (6.1).

Figure 2 makes this concrete. Every direction costs the same step, but they pay differently, and the payoff sweeps through one cosine as the direction rotates. Nothing about the gradient was *chosen* to point uphill; it falls out of the dot product.

The other picture: the graph and its tangent plane

Everything so far has lived in the *domain*: ∇f is an arrow drawn on the contour map, in the xy -plane, at the point (x_0, y_0) . But there is a second picture of the same function — its **graph**, the surface $z = f(x, y)$ sitting in three-dimensional space — and near a point that surface is well approximated by its **tangent plane**. Linearising f about (x_0, y_0) gives that plane directly:

$$z = f(x_0, y_0) + f_x(x_0, y_0)(x - x_0) + f_y(x_0, y_0)(y - y_0). \quad (3.6)$$

Move everything to one side and the plane reads $f_x(x - x_0) + f_y(y - y_0) - (z - z_0) = 0$, which is $\mathbf{n} \cdot (\mathbf{r} - \mathbf{r}_0) = 0$ with

$$\mathbf{n} = (f_x, f_y, -1). \quad (3.7)$$

So the tangent plane's **normal** — and, equivalently, the coefficients of x , y and z when the plane is written $ax + by + cz = d$ — is $(f_x, f_y, -1)$.

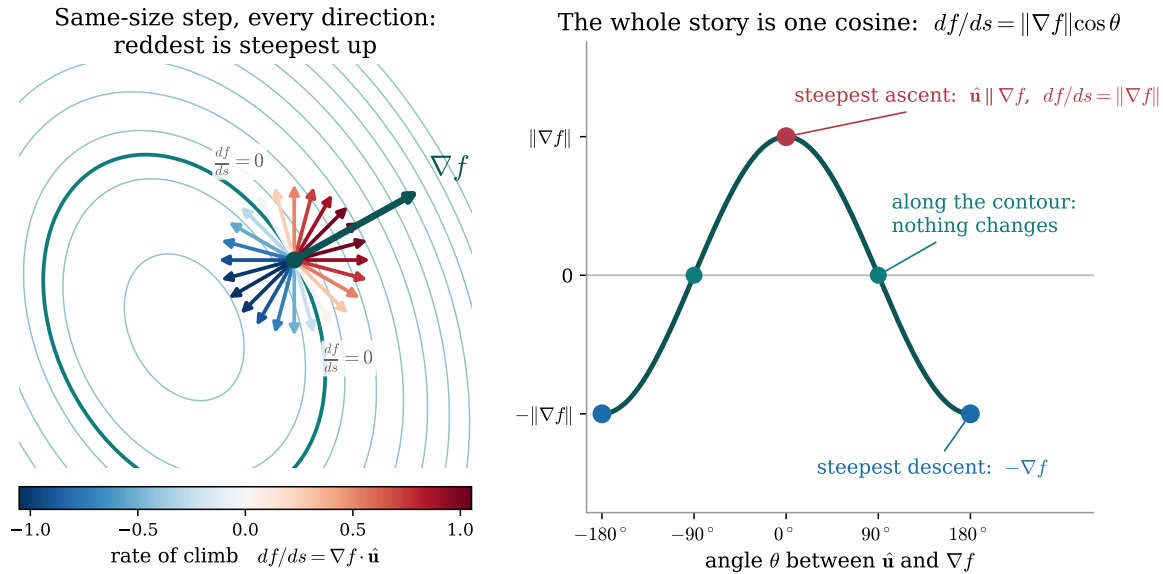


Figure 2: Left: from one point, twenty-four unit steps of identical length, coloured by the rate of climb $\nabla f \cdot \hat{\mathbf{u}}$ each one buys. The reddest points along ∇f ; the two directions tangent to the contour (the bold curve) buy nothing. Right: the same information as a graph. The rate is $\|\nabla f\| \cos \theta$ — one cosine, whose maximum, zeros and minimum are steepest ascent, the contour directions, and steepest descent.

Where does the -1 come from? It is the same theorem as before, one dimension up. Define a function of *three* variables,

$$F(x, y, z) = f(x, y) - z,$$

so that the graph of f is exactly the level set $F = 0$. We already know that a gradient is perpendicular to the level sets of its own function; applied to F that rule gives

$$\nabla F = \left(\frac{\partial F}{\partial x}, \frac{\partial F}{\partial y}, \frac{\partial F}{\partial z} \right) = (f_x, f_y, -1), \quad (3.8)$$

perpendicular to the surface $F = 0$. The -1 is nothing more exotic than $\partial F / \partial z$: the rate at which F changes as you move straight up. The two-dimensional statement “ ∇f is perpendicular to the contours of f ” and the three-dimensional statement “ $(f_x, f_y, -1)$ is normal to the graph of f ” are one theorem applied to two different functions.

The gradient, the plane, and its coefficients

Let $f(x, y) = x^2 + xy$ at the point $(1, 2)$. Then $f_x = 2x + y = 4$, $f_y = x = 1$ and $f(1, 2) = 3$, so $\nabla f = (4, 1)$ and $\|\nabla f\| = \sqrt{17} \approx 4.12$: from $(1, 2)$ the steepest climb is along $(4, 1)$, at a rate of $\sqrt{17}$ per unit distance, and along $(-1, 4)$ — the contour direction — f does not change at all. The tangent plane (3.6) is

$$z = 3 + 4(x - 1) + 1(y - 2) = 4x + y - 3, \quad \text{i.e.} \quad 4x + y - z = 3,$$

whose coefficients are $(4, 1, -1) = (f_x, f_y, -1)$, exactly (3.7).

Three vectors that look alike and are not

This is where the confusion usually sits, because the same two numbers f_x, f_y appear in three different objects. Keep them apart by asking *which space* each lives in:

- $\nabla f = (f_x, f_y)$ lives in the **domain**, the xy -plane — the contour map. It points uphill and its length is the slope. It is not a vector in space and it is not tangent to the surface.
- $\mathbf{n} = (f_x, f_y, -1)$ lives in xyz -space and is **perpendicular to the surface**. It is not a direction you can walk in: step along it and you leave the surface immediately.
- $\mathbf{t} \propto (f_x, f_y, \|\nabla f\|^2)$ also lives in space, is **tangent to the surface**, and is the steepest ascent as a hiker on the surface would actually walk it. (Check: $\mathbf{n} \cdot \mathbf{t} = \|\nabla f\|^2 - \|\nabla f\|^2 = 0$.)

What ties them together is the pair of dashed drop lines in Figure 3: drop either three-dimensional vector straight down onto the xy -plane and its shadow points along ∇f . A useful sanity check on the sign: the *upward* normal is $(-f_x, -f_y, +1)$, whose shadow is $-\nabla f$ — it leans *downhill*. A hillside rising to the east has its upward normal tilted to the west, which is exactly why the normal that carries $+\nabla f$ in its first two slots is the one pointing *down* through the surface.

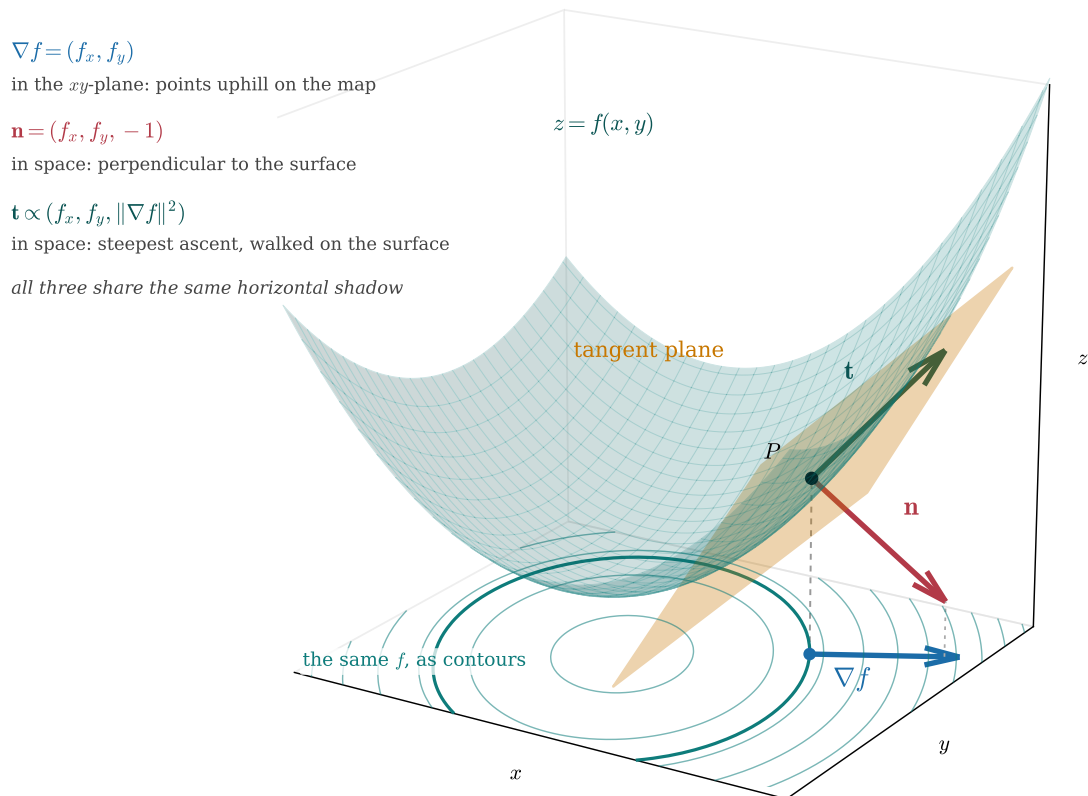


Figure 3: One function, two pictures. The surface $z = f(x, y)$ with its tangent plane at P , and on the floor the contour map of the same f with the same point marked. The normal $\mathbf{n} = (f_x, f_y, -1)$ leaves the surface perpendicularly; $\mathbf{t}$ is the steepest climb along the surface; ∇f is the two-dimensional gradient on the map. The dashed drop lines show what they have in common — one horizontal shadow, pointing along ∇f .

3.4 The Jacobian

For a vector-valued function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, the matrix of all first partials is the **Jacobian**,

$$\mathbf{J} = \frac{\partial f}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}, \quad (3.9)$$

the best linear approximation of f near a point: $f(\mathbf{x} + \delta) \approx f(\mathbf{x}) + \mathbf{J}\delta$.

How good is “best”? Take a step $\delta = h\mathbf{u}$ of length h . The constant guess $f(\mathbf{x})$ is wrong by $O(h)$, because it ignores the motion altogether. The linear guess $f(\mathbf{x}) + \mathbf{J}\delta$ is wrong by only $O(h^2)$ — halve the step and the error *quarters*; what is left over is the second-order Taylor term of Section 4.3. Any other matrix $\mathbf{M}$ leaves an error $(\mathbf{J} - \mathbf{M})\delta$ that is still $O(h)$, so $\mathbf{J}$ is the only linear map with this property — that is the precise sense of “best”. The lab measures both slopes on a two-link arm.

Key idea

This is exactly the manipulator **Jacobian** of the Robotics & Bionics unit, where f is the forward kinematics mapping joint angles to end-effector position, and $\mathbf{J}$ relates joint velocities to end-effector velocity, $\dot{\mathbf{x}} = \mathbf{J}\dot{\mathbf{q}}$. The same object linearises a robot’s motion and a network’s layer. The gradient is just the Jacobian of a scalar function ($m = 1$) — which is why (3.3) could be written as a row times a column.

3.5 The Hessian

The matrix of second partial derivatives is the **Hessian** $\mathbf{H}$; it describes **curvature**. It is symmetric (mixed partials commute), so by the spectral theorem of Module 2 its eigenvalues are real: all positive means a bowl (a local minimum), mixed signs means a saddle. Curvature governs how hard a function is to optimise, and underlies second-order methods.

Exercises

1. Differentiate $f(x) = \frac{1}{2}(wx - y)^2$ with respect to x (treat w, y as constants) using the chain rule.
2. For $f(x, y) = x^2 + 3xy + y^2$, compute ∇f and evaluate it at $(1, 2)$. In which direction does f increase fastest there?
3. A particle moves along $\mathbf{r}(t) = (t, t^2, 0)^\top$ through the potential $V(x, y, z) = x^2 + y$. Compute dV/dt two ways — by substituting the path into V and differentiating, and by $\nabla V \cdot \dot{\mathbf{r}}$ — and check they agree.
4. For the same V at the point $(1, 1, 0)$, find a direction $d\mathbf{r}$ along which $dV = 0$. What is that direction geometrically?
5. For $f(x, y) = x^2 - 2y^2$ at $(2, 1)$: compute ∇f , write the tangent plane to $z = f(x, y)$ in the form $ax + by + cz = d$, and check that (a, b, c) is $(f_x, f_y, -1)$.
6. Using (3.5), in which directions is the rate of climb exactly *half* the steepest rate? (No calculation needed beyond a cosine.)
7. Show that the upward normal $(-f_x, -f_y, 1)$ has a horizontal shadow pointing *downhill*, and say in one sentence why that is the right answer for a hillside.

8. Write the Jacobian of the 2-link planar arm forward kinematics $x = \ell_1 \cos \theta_1 + \ell_2 \cos(\theta_1 + \theta_2)$, $y = \ell_1 \sin \theta_1 + \ell_2 \sin(\theta_1 + \theta_2)$ with respect to (θ_1, θ_2) , and compare with the Robotics unit.
9. State why the central finite difference is more accurate than the forward one.

4 Taylor Expansions: Local Polynomial Models

Every tool so far has made the same move: replace a function, near a point, by something simpler. The derivative replaced f by a line; the tangent plane (3.6) replaced a surface by a plane; the total differential (3.4) did the same thing in differential form. **Taylor's theorem** is that move stated once and for all, to any order, with an honest error term — and it is the reason optimisation algorithms look the way they do.

4.1 One variable

Near a point x , a function is approximated by the polynomial that matches its value and its first n derivatives there:

$$f(x+h) = f(x) + f'(x)h + \frac{f''(x)}{2!}h^2 + \frac{f'''(x)}{3!}h^3 + \cdots + \frac{f^{(n)}(x)}{n!}h^n + R_n, \quad (4.1)$$

with a remainder $R_n = \frac{f^{(n+1)}(\xi)}{(n+1)!}h^{n+1}$ for some ξ between x and $x+h$ — so truncating after the h^n term costs $O(h^{n+1})$. Each order adds one feature of the shape: order 0 is a flat line at the right height, order 1 tilts it to the right slope (that is the tangent line, and (1.1) rearranged), order 2 bends it to the right curvature.

The catch is in the word *near*. A Taylor polynomial is a local object: it is built from derivatives at one point and knows nothing about what the function does elsewhere. Figure 4 shows $\ln(1+x)$, where the polynomials are excellent close to 0 and worthless beyond $|x| > 1$.

4.2 What it says about finite differences

Section 2 asserted that the central difference beats the forward one. Taylor is the proof. Expand in both directions,

$$f(x \pm h) = f(x) \pm f'(x)h + \frac{f''(x)}{2}h^2 \pm \frac{f'''(x)}{6}h^3 + \cdots,$$

and assemble the two estimators:

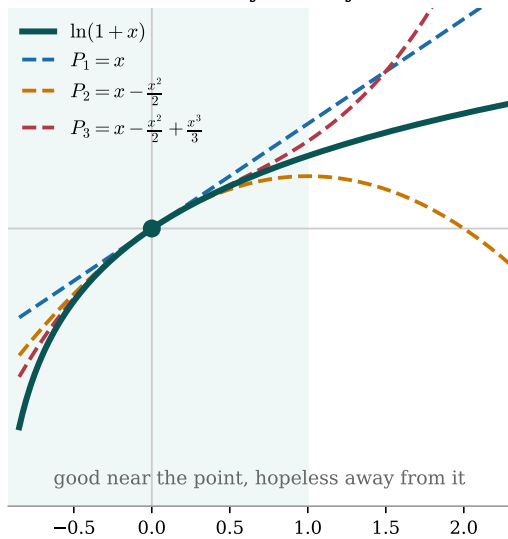
$$\frac{f(x+h) - f(x-h)}{2h} = f'(x) + \frac{h^2}{6}f'''(x) + O(h^4), \quad (4.2)$$

$$\frac{f(x+h) - f(x)}{h} = f'(x) + \frac{h}{2}f''(x) + O(h^2), \quad (4.3)$$

In the central version the even-order terms cancel by symmetry, killing the f'' term entirely and leaving an error one order smaller. That is the whole story: not a numerical accident, just which terms survive a subtraction.

Round-off supplies the other half of the trade-off. In floating point $f(x+h)$ and $f(x-h)$ agree in their leading digits, so subtracting them throws those digits away and leaves an error growing like ε/h as h shrinks. Truncation falls with h and round-off rises as h falls, so the total has a minimum — the “sweet spot” of Section 2, visible as the V in the right panel of Figure 4.

Matching more derivatives buys accuracy,
but only nearby



The same expansion, cashed out:
why the central difference wins

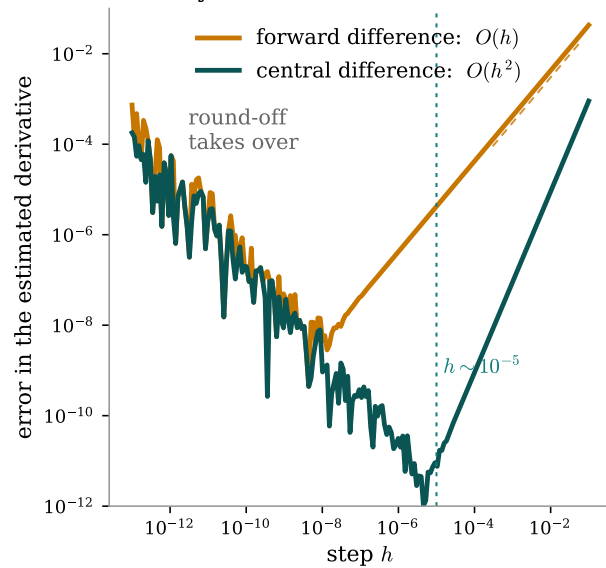


Figure 4: Left: $\ln(1+x)$ and its Taylor polynomials about 0. Each extra order hugs the curve a little longer, but all of them leave it — an expansion about a point is a statement about a neighbourhood of that point, not about the whole function. Right: the finite differences of Section 2, measured. The error falls with the slope Taylor predicts, $O(h)$ for the forward formula and $O(h^2)$ for the central one, until round-off takes over below $h \approx 10^{-8}$; the best accuracy for the central formula lands near $h \sim 10^{-5}$, exactly the rule of thumb quoted earlier.

4.3 Many variables

The same three terms survive the move to a vector argument, as a scalar, a vector and a matrix:

$$f(\mathbf{x} + \delta) = f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \delta + \frac{1}{2} \delta^\top \mathbf{H}(\mathbf{x}) \delta + O(\|\delta\|^3). \quad (4.4)$$

Nothing here is new — it is the whole of Section 3 in one line. Keeping the first two terms and setting $n = 2$ gives exactly the tangent plane (3.6), and in differential form the same two terms are the total differential (3.4). The third term is the Hessian of Section 3.5: its eigenvalues decide whether the local model is a bowl, a saddle, or a ridge.

Key idea

Optimisation is a choice of where to truncate (4.4). Keep only the linear term and the model has no minimum of its own — it just slopes — so the best you can do is move against the slope by a distance you choose yourself: that is gradient descent (Section 6), and the step size η is the price of truncating early. Keep the quadratic term and the model *does* have a minimum; setting its gradient $\nabla f + \mathbf{H}\delta$ to zero gives

$$\delta = -\mathbf{H}^{-1} \nabla f,$$

Newton's method, which chooses its own step and lands exactly on the minimum of a genuinely quadratic loss in one move. The gap between the two is where practical optimisation lives: if the Hessian's eigenvalues span a ratio $\kappa = \lambda_{\max}/\lambda_{\min}$, stability forces $\eta < 2/\lambda_{\max}$ (Section 6.1), and the flattest direction then crawls κ times slower than the stiffest — the ravine that momentum and Adam exist to fix, since forming $\mathbf{H}$ itself ($n \times n$ with n in the millions) is out of the question.

Exercises

1. Write the second-order Taylor polynomial of $\ln(1+x)$ about 0 and use it to estimate $\ln(1.1)$. Compare with the true value, and check that the error is about the size the $O(h^3)$ remainder predicts.
2. Derive (4.2) and (4.3) by expanding $f(x+h)$ and $f(x-h)$, and say in one sentence which symmetry makes the f'' term vanish in the central formula.
3. Write the Taylor expansion of $f(x, y) = x^2 + 3xy + y^2$ about $(0, 0)$. Why does it stop after the quadratic term, exactly?
4. A quadratic loss has Hessian eigenvalues 1 and 100. Show that gradient descent with a fixed η diverges unless $\eta < 0.02$, and explain why the flat direction then needs roughly a hundred times as many steps as the steep one.

5 The Chain Rule, Computational Graphs and Backpropagation

A neural network evaluates a long composition of simple operations. Draw it as a **computational graph** — nodes are operations, edges carry values forward. To get the gradient of the final loss with respect to every parameter, apply the chain rule **backwards** through the graph, multiplying local derivatives along the way. That reverse sweep is **backpropagation**.

Backprop on a tiny graph

Let $L = (ab + c)^2$ with $a = 2$, $b = 3$, $c = 1$. Forward: $u = ab = 6$, $v = u + c = 7$, $L = v^2 = 49$. Backward, using local derivatives:

$$\begin{aligned} \frac{\partial L}{\partial v} = 2v = 14, \quad \frac{\partial L}{\partial u} = \frac{\partial L}{\partial v} \cdot 1 = 14, \quad \frac{\partial L}{\partial a} = \frac{\partial L}{\partial u} \cdot b = 42, \\ \frac{\partial L}{\partial b} = \frac{\partial L}{\partial u} \cdot a = 28, \quad \frac{\partial L}{\partial c} = \frac{\partial L}{\partial v} \cdot 1 = 14. \end{aligned}$$

Each gradient is a product of local derivatives along the path from L back to the input — the chain rule, bookkept by the graph. A framework does this automatically for graphs with millions of nodes.

5.1 A real layer: differentiating with respect to a matrix

In that example a , b and c were single numbers. A network layer is $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$ with $\mathbf{W}$ an $m \times n$ *matrix* of parameters, so the object we need is $\partial L / \partial \mathbf{W}$: the derivative of one scalar with respect to a whole matrix. Write $\boldsymbol{\delta} = \partial L / \partial \mathbf{z}$ for the gradient that backprop has already delivered to this layer's output. Since $z_i = \sum_j W_{ij}x_j$, the parameter W_{ij} affects only z_i , and it does so with weight x_j ; the chain rule therefore gives $\partial L / \partial W_{ij} = \delta_i x_j$, which assembled into a matrix is an outer product:

$$\frac{\partial L}{\partial \mathbf{W}} = \boldsymbol{\delta} \mathbf{x}^\top, \quad \frac{\partial L}{\partial \mathbf{x}} = \mathbf{W}^\top \boldsymbol{\delta}. \quad (5.1)$$

The second expression is what gets passed further back, and it is the reason transposes appear everywhere in backprop derivations: the forward pass sends information through $\mathbf{W}$, the backward pass sends it through $\mathbf{W}^\top$.

Key idea

The shape rule that saves hours. The gradient of a scalar with respect to *anything* has the same shape as that thing: $\partial L / \partial \mathbf{W}$ is $m \times n$ exactly like $\mathbf{W}$, $\partial L / \partial \mathbf{x}$ is n like $\mathbf{x}$. Check it in (5.1): $\boldsymbol{\delta}$ is $m \times 1$ and $\mathbf{x}^\top$ is $1 \times n$, so the outer product is $m \times n$; and $\mathbf{W}^\top$ is $n \times m$ against an $m \times 1$ vector, giving $n \times 1$. When a hand-derived gradient does not match the shape of its parameter, the derivation is wrong — and this one check catches most such errors before any code runs.

5.2 Why depth is hard: vanishing and exploding gradients

Backpropagation through L layers multiplies L local factors together. That single sentence, taken seriously, predicts the central difficulty of deep learning. If each factor has a typical magnitude ρ , the gradient arriving at the earliest layer is of size ρ^L — exponential in the depth. There is no comfortable middle: ρ slightly below one and the signal dies, ρ slightly above one and it blows up.

Figure 5 puts numbers on it. A sigmoid layer is the classic offender: $\sigma' = \sigma(1 - \sigma)$ never exceeds $\frac{1}{4}$, and away from the origin it is far smaller because the unit is saturated. Even at its very best, ten stacked sigmoid layers shrink the gradient by $(\frac{1}{4})^{10} \approx 10^{-6}$, so the early layers of a deep network learn roughly a million times more slowly than the late ones — if they learn at all.

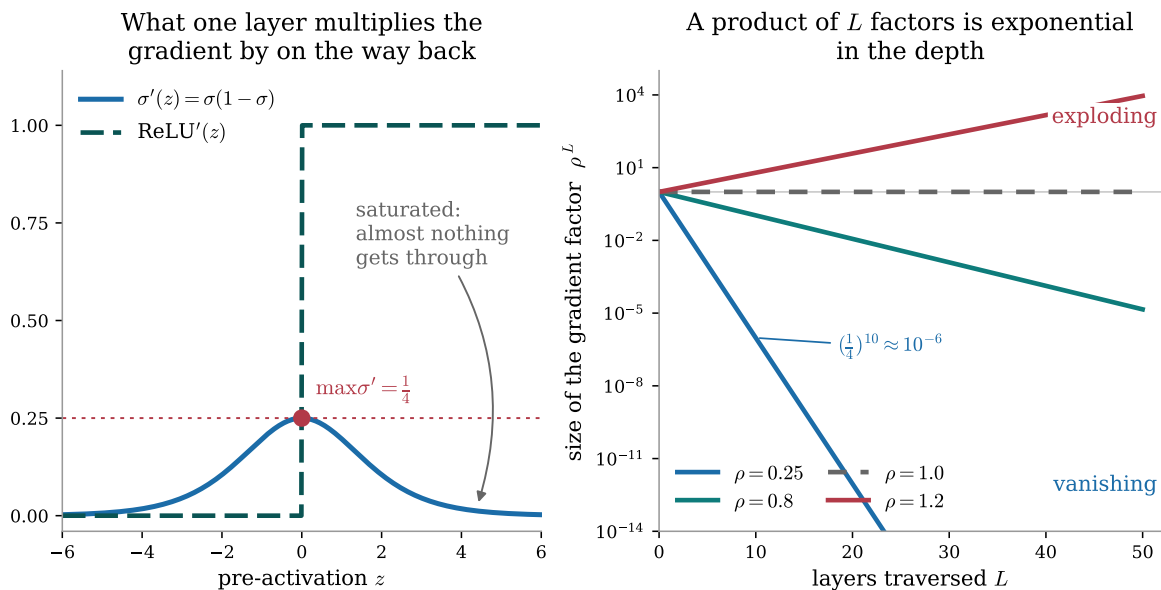


Figure 5: Left: what a single layer multiplies the returning gradient by. The logistic derivative peaks at $\frac{1}{4}$ and collapses once the unit saturates, while a ReLU passes the gradient through unchanged wherever it is active. Right: the same factor applied L times. Because the chain rule multiplies, the depth enters as an exponent, and only $\rho = 1$ is neutral — everything else vanishes or explodes.

Key idea

Read Chapter 11 of Géron with (5.1) in mind and its remedies stop looking like a list of tricks. **ReLU** replaces a derivative of at most $\frac{1}{4}$ with one of exactly 1 on the active side. **Careful initialisation** (Glorot, He) chooses the starting scale of $\mathbf{W}$ so that $\rho \approx 1$ at the outset rather than by luck. **Batch normalisation** keeps the pre-activations in the range

where derivatives are healthy instead of saturated. **Gradient clipping** caps the exploding case directly, and **residual (skip) connections** add a path whose local derivative is 1, so there is always a route back that the product cannot kill. Each one is an attack on the same exponent.

5.3 Backprop is a product of Jacobians — and why it runs backwards

Section 5.1 treated one layer. Stack three, $\mathbf{h}_k = \phi(\mathbf{z}_k)$ with $\mathbf{z}_k = \mathbf{W}_k \mathbf{h}_{k-1}$ and $\mathbf{h}_0 = \mathbf{x}$, each of width n , and put a scalar loss $L(\mathbf{h}_3)$ on top. Each layer is a map $\mathbb{R}^n \rightarrow \mathbb{R}^n$ with its own Jacobian (Section 3.4), $\mathbf{J}_k = \partial \mathbf{h}_k / \partial \mathbf{h}_{k-1} = \text{diag}(\phi'(\mathbf{z}_k)) \mathbf{W}_k$, and the chain rule for a composition is the **product** of the Jacobians:

$$\frac{\partial L}{\partial \mathbf{x}} = \underbrace{\mathbf{g}^\top}_{1 \times n} \mathbf{J}_3 \mathbf{J}_2 \mathbf{J}_1, \quad \mathbf{g}^\top = \frac{\partial L}{\partial \mathbf{h}_3}. \quad (5.2)$$

This is Module 2's *composition is matrix multiplication*, seen through calculus. It has a consequence that is easy to miss: matrix multiplication is **associative**, so (5.2) can be evaluated in either order — the same numbers, at very different cost (Figure 6).

- **From the loss end**, $((\mathbf{g}^\top \mathbf{J}_3) \mathbf{J}_2) \mathbf{J}_1$. Every intermediate is a single row, and each row-times-matrix step costs n^2 multiply-adds: Ln^2 in total for L layers. This is **reverse mode**, and it *is* backpropagation. Since $\mathbf{g}^\top \mathbf{J}_k = (\mathbf{J}_k^\top \mathbf{g})^\top$ and $\mathbf{J}_k^\top = \mathbf{W}_k^\top \text{diag}(\phi'(\mathbf{z}_k))$, each step is “multiply elementwise by ϕ' , then by $\mathbf{W}_k^\top$ ” — exactly the backward pass of (5.1).
- **From the input end**, $\mathbf{g}^\top (\mathbf{J}_3 (\mathbf{J}_2 \mathbf{J}_1))$. Every intermediate is a full $n \times n$ matrix and each product costs n^3 : $(L-1)n^3 + n^2$ in total. This is **forward mode**.

The ratio is roughly the width n . For ten layers of width 512 that is 2.6×10^6 multiply-adds against 1.2×10^9 — about 460 times the work for an identical gradient.

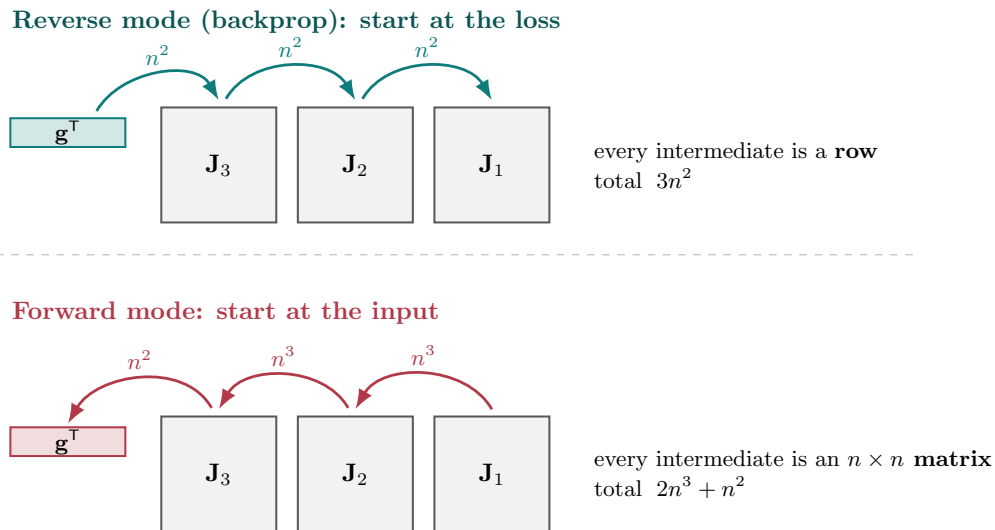


Figure 6: One chain of Jacobians, two association orders. Starting from the loss, the running product is always a row and each step costs n^2 ; starting from the input, it is an $n \times n$ matrix and each step costs n^3 . Same gradient, a factor of about n in work.

Key idea

Start multiplying from the thin end of the chain. The rule is about shape, not direction. Training has *one* scalar loss and millions of parameters, so the chain begins with a single row on the output side and reverse mode wins by a factor of the width — which is why PyTorch’s `.backward()` is called on the loss. When a model has few inputs and many outputs — say the sensitivity of 200 predicted plasma concentrations to a single dose — the thin end is on the input side and forward mode wins instead. Autodiff libraries offer both: vector–Jacobian products for reverse mode, Jacobian–vector products for forward mode (`vjp` and `jvp` in PyTorch’s `torch.func` and in JAX).

Counting operations versus timing them

When the lab times both orders at width 256 the gap is roughly an order of magnitude, well short of the $\approx 220\times$ the count predicts: optimised linear-algebra libraries (BLAS) run matrix–matrix products far more efficiently per operation than vector–matrix ones. The count is what grows with n , though, and at the widths of real networks no library closes it. Reverse mode has one real price: to form each $\mathbf{J}_k$ on the way back it must have kept the forward activations $\mathbf{z}_k$, which is why training uses far more memory than prediction.

Exercises

1. A layer has $\mathbf{W}$ of shape 3×4 and input $\mathbf{x}$ of length 4. Give the shapes of $\mathbf{z}$, $\boldsymbol{\delta}$, $\partial L / \partial \mathbf{W}$ and $\partial L / \partial \mathbf{x}$, and check each against (5.1).
2. A network has 20 sigmoid layers, every one of them at its most generous ($\sigma' = \frac{1}{4}$). By what factor is the gradient reaching the first layer smaller than the one at the last? Repeat the count for ReLU units that are all active.
3. Suppose you trained a classifier with squared error through a sigmoid, $L = \frac{1}{2}(\hat{y} - y)^2$ with $\hat{y} = \sigma(z)$. Show that $\partial L / \partial z = (\hat{y} - y) \sigma'(z)$, and explain why a confidently *wrong* prediction then produces almost no gradient — while the log-loss of (1.2) produces the largest one. This is why classifiers are trained with cross-entropy and not squared error.
4. A pharmacokinetic model’s sensitivity chain $\mathbf{J}_3 \mathbf{J}_2 \mathbf{J}_1$ has shapes 200×50 , 50×50 and 50×1 (one dose in, 200 predicted concentrations out). Using *abc* multiply-adds for an $(a \times b)(b \times c)$ product, count the cost of both association orders. Which wins, and why does that not contradict the fact that backprop runs backwards?
5. Show that $\mathbf{g}^\top \mathbf{J} = (\mathbf{J}^\top \mathbf{g})^\top$ and, with $\mathbf{J} = \text{diag}(\phi') \mathbf{W}$, that one reverse-mode step multiplies $\mathbf{g}$ elementwise by ϕ' and then by $\mathbf{W}^\top$.

6 Optimisation: Gradient Descent

To minimise a loss $L(\mathbf{w})$ we repeatedly step **downhill**, opposite the gradient:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L(\mathbf{w}), \quad (6.1)$$

where the **learning rate** η sets the step size. Too small and training crawls; too large and it overshoots or diverges. This single update — in batch, mini-batch or stochastic form — trains nearly every model in Géron’s book.

Why it works: Taylor. Truncate (4.4) after the linear term. The model $L(\mathbf{w} + \delta) \approx L(\mathbf{w}) + \nabla L^\top \delta$ is decreased most by stepping along $-\nabla L$, which is exactly (6.1) — the total differential (3.4) read as advice about which way to step. Everything that makes a single fixed η

awkward, and every second-order alternative to it, lives in the quadratic term that this truncation threw away (Section 4).

Convex vs non-convex. A convex loss (a single bowl, e.g. linear/logistic regression) has one global minimum that gradient descent reaches. Deep networks are non-convex — many minima and saddles — yet gradient descent still finds good solutions in practice. You will watch it descend in the lab; the next subsection shows that the largest usable η need not be found by trial and error.

6.1 The learning-rate law

Near a minimum $\mathbf{w}^*$ the gradient vanishes, so the Taylor model (4.4) is a bowl, $L(\mathbf{w}) \approx L^* + \frac{1}{2} \mathbf{e}^\top \mathbf{H} \mathbf{e}$ with $\mathbf{e} = \mathbf{w} - \mathbf{w}^*$, whose gradient is $\nabla L \approx \mathbf{H} \mathbf{e}$. One step of (6.1) therefore acts on the error as a matrix:

$$\mathbf{e} \leftarrow (\mathbf{I} - \eta \mathbf{H}) \mathbf{e}. \quad (6.2)$$

$\mathbf{H}$ is symmetric (Section 3.5), so it has an orthonormal eigenbasis, $\mathbf{H} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^\top$. Describe the error in that basis, $\mathbf{e} = \mathbf{Q} \mathbf{c}$ — the *easy-basis* move of Module 2, §8.2 — and $\mathbf{I} - \eta \mathbf{H}$ becomes diagonal. Gradient descent splits into independent one-variable recursions, one per eigen-direction:

$$c_i \leftarrow (1 - \eta \lambda_i) c_i, \quad \text{so} \quad c_i^{(k)} = (1 - \eta \lambda_i)^k c_i^{(0)}. \quad (6.3)$$

The learning-rate law

On a quadratic loss with positive-definite Hessian, gradient descent converges from every starting point if and only if every factor satisfies $|1 - \eta \lambda_i| < 1$, that is

$$0 < \eta < \eta^* = \frac{2}{\lambda_{\max}}.$$

Above η^* the stiffest mode grows by $|1 - \eta \lambda_{\max}| > 1$ every step and the iterates diverge. Below it every mode shrinks, but the slowest one only by $\max_i |1 - \eta \lambda_i|$ per step; even at the best fixed step, $\eta = 2/(\lambda_{\max} + \lambda_{\min})$, that factor is $(\kappa - 1)/(\kappa + 1) \approx 1 - 2/\kappa$, with $\kappa = \lambda_{\max}/\lambda_{\min}$ the **condition number**.

Figure 7 tests both halves of the statement on a least-squares fit, whose loss is exactly quadratic. The threshold is sharp: a four percent change in η is the difference between convergence and divergence. And in the eigenbasis each mode follows (6.3) to the last digit, independent of the others.

Why κ matters. That problem is well conditioned, $\kappa \approx 1.5$. The breast-cancer classifier of the lab is not: its two features, mean radius and mean area, are nearly the same measurement (correlation 0.99), so shifting weight from one to the other barely changes the loss. The contours form a long, narrow ravine with $\kappa \approx 60$; the step size is capped by the steep walls, and progress along the flat floor crawls. Standardising features (Géron, Chapter 4) removes the part of κ that comes from mismatched units; the part that comes from nearly redundant features remains, and it is what regularisation and PCA address.

The law is local

For a loss that is not quadratic, $\mathbf{H}$ changes with $\mathbf{w}$ and so does η^* . For logistic regression $\mathbf{H} = \mathbf{X}^\top \text{diag}(p(1-p)) \mathbf{X}/m$. At $\mathbf{w} = \mathbf{0}$ every p is $\frac{1}{2}$, where $p(1-p)$ is at its maximum $\frac{1}{4}$; as the model grows confident, $p \rightarrow 0$ or 1 and the curvature falls. On the lab's classifier, η^* read at the start is about 4, read at the optimum about 22 — and the loss in fact first stops decreasing between $\eta = 22$ and 25. Deep networks change curvature throughout training, which is one reason for the learning-rate schedules of Géron's Chapter 11.

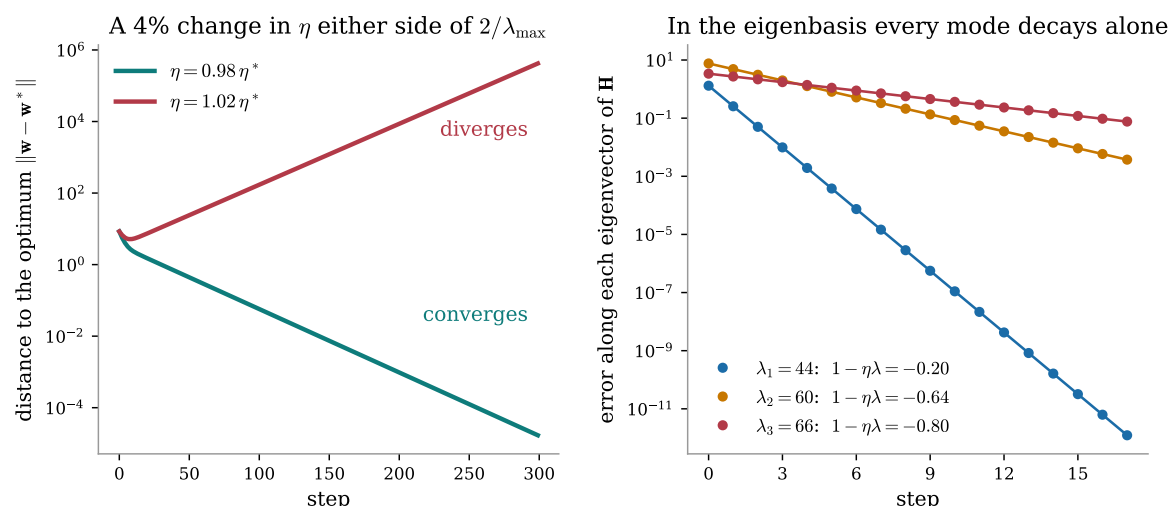


Figure 7: Gradient descent on a three-parameter least-squares problem with Hessian eigenvalues 44, 60 and 66, so $\eta^* = 2/66 \approx 0.030$. Left: just below and just above η^* . Right: at $\eta = 0.9\eta^*$, the error along each eigenvector (dots) against the prediction $|1 - \eta\lambda_i|^k$ (lines). Note that the stiffest mode, whose factor is closest to -1 , flips sign every step and is the slowest to settle — it is the one that sets the limit.

Exercises

1. $L(w) = (w - 5)^2$ has Hessian 2. What is η^* ? Starting from $w = 0$, describe the iterates for $\eta = 0.5$, 1 and 1.5, and match each to a factor $1 - \eta\lambda$.
2. The problem of Figure 7 has Hessian eigenvalues 44, 60 and 66. At $\eta = 0.9\eta^*$, compute the three factors $1 - \eta\lambda_i$ and check them against the figure's legend. Which mode is slowest to converge, and why is it not the one with the smallest eigenvalue?
3. In one sentence, why is η^* for logistic regression smaller when read at $\mathbf{w} = \mathbf{0}$ than at the optimum?

7 Automatic Differentiation and PyTorch

How does a framework get exact gradients of arbitrary code? Not by finite differences (too inaccurate) nor by symbolic algebra (formulas explode), but by **automatic differentiation** (autodiff): it records each elementary operation and applies the chain rule numerically, exactly, in one reverse pass — precisely the backprop of Section 5, generalised: reverse mode, the thin-end order of Section 5.3. This is what PyTorch's `autograd` does.

Key idea

In PyTorch you mark a tensor with `requires_grad=True`; every operation is recorded; calling `.backward()` on a scalar loss fills each tensor's `.grad` with the exact derivative. A training loop is then just: compute loss, `backward()`, step by (6.1), zero the gradients, repeat. The lab shows the same gradient three ways — by hand, by finite difference, and by autograd — agreeing to machine precision. This mirrors Géron's autodiff appendix and is the foundation of every training loop in Chapters 9–11.

Exercises

1. Backprop by hand through $L = (2a + b)^2$ at $a = 1, b = 3$: give $\partial L / \partial a$ and $\partial L / \partial b$.
2. One step of gradient descent on $L(w) = (w - 5)^2$ from $w = 0$ with $\eta = 0.1$: what is the new w ? Toward what value will it converge?

8 The Lab, and Where This Leads in Géron

Key idea

Companion Colab notebook. Part 1 ships one self-contained notebook:

- `m3_derivatives_gradients` — derivatives and the chain rule; numerical derivatives and gradient checking; partials, the gradient and the Jacobian with its $O(h^2)$ linearisation error; backprop as a product of Jacobians, timed in both orders; gradient descent from scratch on a biomedical loss, with the learning rate predicted from the Hessian; and the same gradient by hand, by finite difference and by PyTorch autograd.

It is a superset of Géron's `math_differential_calculus` and autodiff material, on biomedical examples, with exercises and solutions. Part 2 — integration and the PID controller — ships its own companion notebook, taught later alongside the Control Theory unit.

Where this unlocks the book. Gradient descent is Chapter 4 (training linear models); autodiff and the gradient power Chapters 9–11 (training neural nets and backprop); the Jacobian is the Robotics unit's. Integration and the PID controller's integral term return in Part 2, right before the Control unit.

The one-paragraph summary

The derivative is a rate and a slope; the chain rule differentiates compositions and, run backwards on a computational graph, is backpropagation — which multiplies one factor per layer, so depth makes gradients vanish or explode, and runs backwards because a scalar loss makes the output the thin end of the chain of Jacobians. Every loss worth the name differentiates to *error times input*. The gradient points uphill because the rate of climb is $\|\nabla f\| \cos \theta$, biggest along ∇f — so we train by stepping the opposite way (gradient descent, step size η , safe below $2/\lambda_{\max}$ of the Hessian), and on the graph of f those same two numbers reappear as the tangent plane's normal $(f_x, f_y, -1)$. That plane is the first-order Taylor model, and where you truncate Taylor is what separates gradient descent from Newton's method. Dotted into a displacement the gradient gives the total differential $dV = \nabla V \cdot d\mathbf{r}$; dotted into a velocity, the rate a travelling particle feels. The Jacobian generalises it to vector outputs and is the robot's; the Hessian is curvature. Autodiff computes all of it exactly. Differentiate to learn; integrate to control (Part 2).